

What is NEXT.js

Why it matters



Next.js

Next.js is a popular React framework that enables server-side rendering (SSR) and static site generation (SSG) for React applications.

It provides an easy-to-use and powerful setup for building server-rendered React applications with great performance and SEO benefits.

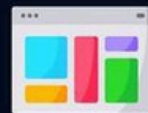


Why it matters:

- Performance: SSG and SSR make pages load faster
- SEO Boost: Server-rendered content is better for search engines
- File-based routing: No React Router needed
- Fullstack features: Build backend routes, fetch data on the server, all in one codebase
- Built-in image optimization and styling support



Installation



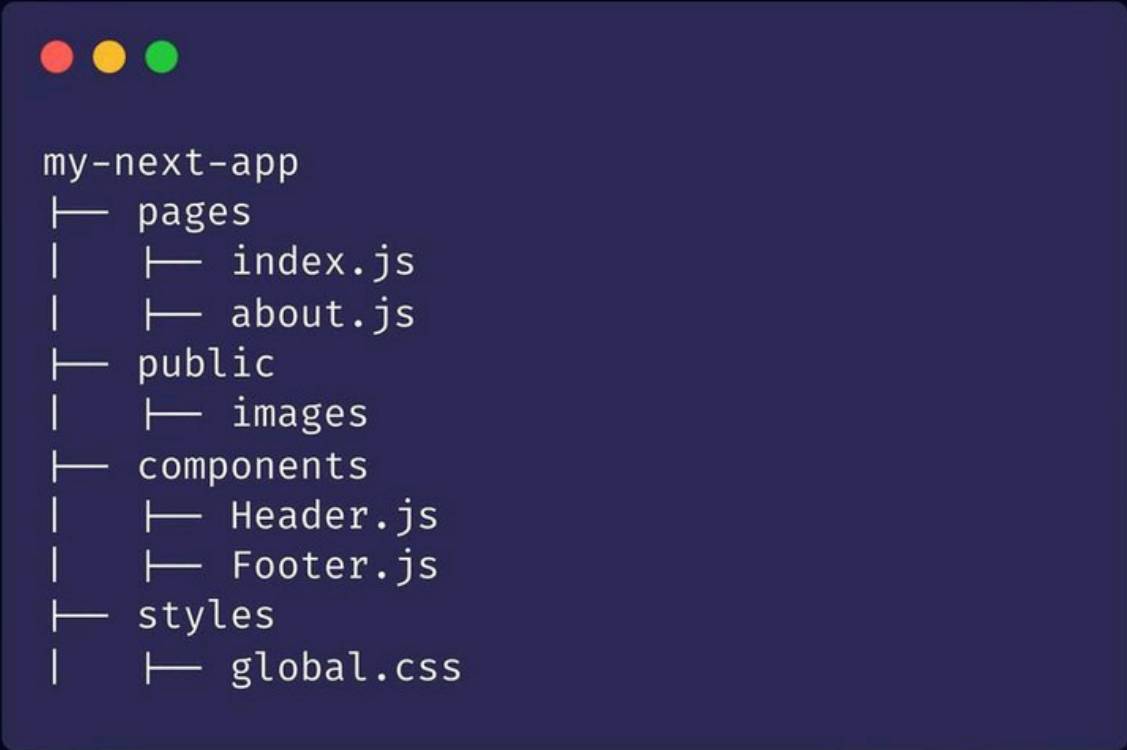
You can create a new Next.js project by using the following command:

```
npx create-next-app my-next-app  
cd my-next-app
```



Basic Structure

Next.js v12 follows a conventional folder structure:



```
my-next-app
├─ pages
│   ├─ index.js
│   └─ about.js
├─ public
│   └─ images
├─ components
│   ├─ Header.js
│   └─ Footer.js
├─ styles
└─ global.css
```

A terminal window with a dark blue background and three colored window control buttons (red, yellow, green) in the top-left corner. The terminal displays the file structure of a Next.js v12 application. The root directory is 'my-next-app'. It contains several subdirectories: 'pages' (with 'index.js' and 'about.js'), 'public' (with 'images'), 'components' (with 'Header.js' and 'Footer.js'), and 'styles'. There is also a 'global.css' file at the root level.



Basic Structure II



- **pages:** Each file inside this folder becomes a route accessible in the application.
- **public:** Contains static assets like images, which can be used in pages.
- **components:** Reusable components used across multiple pages.
- **styles:** Global styles and CSS files.





Basic Page

To create a basic page, simply create a new file inside the pages directory:

```
index.js

// pages/index.js

import React from 'react';

const HomePage = () => {
  return (
    <div>
      <h1>Welcome to Next.js</h1>
      <p>This is a basic Next.js page.</p>
    </div>
  );
};

export default HomePage;
```



Dynamic Routing



Next.js allows you to create dynamic routes using brackets []. For example, to create a dynamic post page:

```
[postId].js

// pages/posts/[postId].js

import React from 'react';
import { useRouter } from 'next/router';

const PostPage = () => {
  const router = useRouter();
  const { postId } = router.query;

  return (
    <div>
      <h1>Post ID: {postId}</h1>
      { /* Fetch and display post details using postId */ }
    </div>
  );
};

export default PostPage;
```



Server-side Rendering (SSR)

Next.js supports server-side rendering, allowing you to fetch data and render pages on the server before sending them to the client:

```
ssr.js

// pages/ssr.js
import React from 'react';

const SSRPage = ({ serverData }) => {
  return (
    <div>
      <h1>Server-Side Rendering</h1>
      <p>Data fetched on the server: {serverData}</p>
    </div>
  );
};

export async function getServerSideProps() {
  const serverData = 'This data was fetched on the server-side.';
  return {
    props: {
      serverData,
    },
  };
}

export default SSRPage;
```



API Routes



Next.js allows you to create API routes, providing a simple way to create custom serverless API endpoints:

```
hello.js

// pages/api/hello.js

export default function handler(req, res) {
  res.status(200).json(
    { message: 'Hello, this is an API endpoint!' }
  );
}
```

